Evaluating Hybrid Recommenders and Post-Hoc Reranking Strategies

Jimena de Prado
University of Antwerp
Student ID: 0259865
Email: Jimena.dePradoGonzalez@student.uantwerpen.be

***Abstract*—This work investigates whether hybrid recommender systems that combine collaborative filtering (CF) and content-based methods (CB) can outperform single-model approaches in terms of both accuracy and fairness using Steam games' dataset. Furthermore, we analyze the impact of post-hoc re-ranking techniques on the results obtained by the hybrid recommender.**

1. Introduction

Platforms such as Steam rely heavily on recommender systems to help users discover video games that match their interests. With thousands of available games, effective recommendations are crucial not only to improve user satisfaction, but also to help smaller titles reach their target audiences rather than being overshadowed by the most popular games.

Traditional recommender systems have been commonly built using either collaborative filtering (CF) or content-based (CB) approaches. While both methods have complementary strengths and weaknesses, neither is sufficient on its own to address all recommendation challenges. To overcome these limitations, hybrid recommender systems have been widely proposed as a way to combine the advantages of both approaches. In this work, we focus on a hybrid model based on Feature-Weighted Linear Stacking (FWLS).

Although hybrid recommender systems often achieve strong accuracy, they may still reinforce popularity bias and reduce exposure for less mainstream items. To address this issue, post-hoc re-ranking strategies have been introduced as a mechanism to modify recommendation lists after score generation. In this paper, we analyze two of these techniques: popularity penalty and Maximal Marginal Relevance (MMR).

December 20, 2025

1.0.1. Evaluation. For the content-based and collaborative filtering models, we split the provided test data into a *test-in* (fold-in) set and a *test-out* set. This allowed us to simulate a recommendation scenario, since submitting every experimental variant to Codabench resulted in long waiting times and failed attempts. Although this split is not part of the original evaluation protocol, it was useful for analysing and comparing the behaviour of these individual models.

For the hybrid recommender and the post-hoc re-ranking strategies, however, we followed the official evaluation setup. In these cases, the final recommendation lists were generated using the complete test-in data, and they were evaluated through Codabench.

Also, I would like to clarify that when submitting the code to the repository, I was unable to upload the datasets because of their size.

2. Content-Based

This implementation is mainly inspired by the content-based filtering chapter presented by Falk, which focuses on extracting information directly from item descriptions instead of relying exclusively on user interaction data.

In this case we chose to implement the topic modeling technique of Latent Dirichlet Allocation (LDA), which requires rich textual information. For this reason, we use the *extended_games* dataset, as it contains detailed metadata for each game.

2.1. Dataset Selection

From the *extended_games* dataset, we select several text-based columns that describe each game, such as the game name, short description, detailed description, and user reviews, together with all available tag columns. The goal is to capture as much semantic information as possible about each game.

To ensure consistency, missing tag values are filled with zeros, while missing text fields are replaced with empty strings. This step is crucial because LDA cannot properly handle missing values.

2.2. Data Preprocessing

Once the relevant fields are selected, we preprocess the text following the standard pipeline described by Falk. First, all text is converted to lowercase. Then, HTML tags and non-alphabetic characters are removed. We also remove common English stopwords, such as "the" or "and", and filter out very short or uninformative tokens.

After preprocessing, all text fields are merged into a single document per game called 'text', as you can see in



Figure 1. Example of latent topics extracted by the LDA model

the code. This results in one consolidated text representation for each item, which matches the expected input format for LDA.

2.3. Topic Modeling with LDA

Latent Dirichlet Allocation (LDA) is a topic modeling algorithm where, rather than attempting to interpret the text directly, it is based on two simple assumptions:

- each document is a mixture of latent topics
- and each topic is itself a mixture of words

As explained by Falk, this process can be understood through an intuitive metaphor. Topics can be seen as hidden "bags" of words, and when a document is generated, words are sampled from different topic bags according to certain probabilities. As a result, a single document may simultaneously reflect multiple topics in different proportions.

In our context, a game description may simultaneously relate to multiple underlying themes, such as action, multiplayer, or fantasy. These topics are not predefined, instead, LDA automatically discovers them based on word co-occurrence patterns in the dataset.

After training the LDA model, each game is represented as a topic distribution vector. This transforms raw textual descriptions into a numerical representation that can be directly compared across items.

2.4. Item Similarity and Recommendation Generation

To connect content information with user preferences, we represent users' behavior using a user–item interaction matrix, where rows correspond to users, columns correspond to games, and each entry indicates whether a user has interacted with a given game. This matrix acts as a bridge between item representations and user histories. Then, using the topic vectors obtained from LDA, we compute an item–item similarity matrix based on cosine similarity, as suggested by Falk.

To generate recommendations, we follow a standard content-based strategy. For each user, we retrieve the games they have already interacted with and identify content-wise similar games using the item–item similarity matrix we had defined. The similarities are aggregated into a score for each candidate game, which is then used to rank items and produce a top-N recommendation list. This approach recommends games that are content-wise similar to those the user already enjoys, based entirely on item descriptions.

TABLE 1. CONTENT-BASED RESULTS (TOPN=20).

Metric	Value	Type
NDCG@20	0.00733	Accuracy
Recall@20	0.01284	Accuracy
Item Gini@20	0.84845	Fairness

2.5. Limitations of the Content-Based Model

The evaluation results for the pure content-based model show relatively low accuracy metrics. This outcome is expected and is also discussed in Falk's work. Content-based recommenders do not benefit from collective user behavior and tend to prioritize similarity over popularity.

As a result, CB models are effective at recommending "more of the same" and at handling cold-start items, but they are not optimized for maximizing accuracy-based metrics. This limitation motivates the introduction of collaborative filtering and hybrid recommender systems in the following sections.

3. Collaborative Filtering

We also implemented a collaborative filtering (CF) model, which focuses exclusively on user behaviour rather than item content. For this, we relied on an ItemKNN model that we had already developed in an earlier assignment and adapted it to fit into the current problem.

3.1. User-Item matrix

The main difference between collaborative filtering and content-based filtering lies on where they get their source of similarity. While content-based methods rely on item descriptions, collaborative filtering gets similarity directly from user interaction patterns.

To model user behaviour, we built a user-item interaction matrix, where rows represent users, columns represent items, and each entry indicates whether a user has interacted with a given item. Concretely, we first define a global user and item universe in our code by taking the union of identifiers appearing on each dataset. We then map the original `user_id` and `item_id` values to indices and construct a sparse matrix where the value 1 corresponds to an interaction. This interaction matrix is the core of the collaborative filtering model, as it captures patterns such as *"users who interacted with this game also interacted with that one"*.

In ItemKNN, two items are considered similar if they are frequently consumed by the same users. Similarity is therefore not based on content, but on shared user behaviour. We compute item-item similarity using cosine similarity. For two items i and j, represented by their interaction vectors $\mathbf{x}_i$ and $\mathbf{x}_j$ (columns of the user-item matrix), cosine similarity is defined as:

$$\mathrm{sim}(i,j) = \frac{\mathbf{x}_i^\top \mathbf{x}_j}{\|\mathbf{x}_i\|_2 \, \|\mathbf{x}_j\|_2}.$$

```
#map internal matrix indices back to the original user_id and item_id values
df_recos["user_id"] = all_users.to_numpy()[df_recos["user_id"].to_numpy()]
df_recos["item_id"] = all_items.to_numpy()[df_recos["item_id"].to_numpy()]
```

Figure 2. Mapping back to the original indices

In this case, the vectors encode user interactions rather than textual features, so cosine similarity measures how often two games are consumed by the same users, instead of how similar their descriptions are.

3.2. K-Nearest Neighbours

Then, we apply a k-nearest neighbours strategy: for each item, only the top-K most similar items are retained, while weaker similarities are discarded.

This results in a sparse item-item similarity matrix that captures the strongest relationships between items based on user interaction data. To get the recommendations, we follow a similar structure to the content-based approach. For each user:

- the set of items previously interacted with by the user is identified
- for each of these items, the K most similar items are retrieved from the item-item similarity matrix
- the similarity scores of these items are added together to obtain a single score for each candidate game,
- and games are then ordered from highest to lowest score to produce the final recommendation list

Formally, the scoring mechanism can be expressed as:

$$\text{scores} = X_{\text{user,item}} \cdot S_{\text{item,item}},$$

where $X_{\text{user,item}}$ is the user-item interaction matrix and $S_{\text{item,item}}$ is the item-item similarity matrix.

3.2.1. Additional Implementation Details. As we explained before, in order to build the interaction matrices efficiently, users and items are mapped to continuous integer indices, which are then used to build sparse matrices for both the training data and the fold-in test interactions.

Initially, this indexing step led to very poor evaluation results. The issue was not caused by the collaborative filtering model itself, but by an incorrect mapping between the internal matrix indices and the original user and item identifiers when generating the recommendation dataframe. As a result, recommendations were evaluated using mismatched user-item pairs, which severely distorted accuracy metrics.

This issue was fixed by mapping the internal matrix indices back to the original user and item identifiers before calculating the results. In our code, we simply translated the internal indices back to the real user and item IDs. Once this mapping was corrected, the collaborative filtering results became consistent and interpretable.

TABLE 2. COLLABORATIVE FILTERING (ITEMKNN) RESULTS (TOPN=20).

Metric	Value	Type
NDCG@20	0.2296	Accuracy
Recall@20	0.3083	Accuracy
Item Gini@20	0.9622	Fairness

3.3. Limitations of Collaborative Filtering Model

After correcting the implementation, collaborative filtering achieves much better results in terms of accuracy, obtaining relatively high values for both NDCG@20 and Recall@20. This indicates that ItemKNN is able to capture meaningful behavioural patterns and generate relevant recommendations.

However, CF performs worse in terms of fairness, as reflected by a high Item Gini value. This suggests that the recommendations are strongly concentrated on a small set of popular items, reinforcing popularity bias. In other words, it tends to favour already popular games and provides limited exposure to less mainstream titles.

As we have already discussed, the content-based model is less dependent on user interaction density and can still recommend items based solely on their textual descriptions. This makes it robust in cold-start scenarios, where new or rarely interacted items be compared to other games, whereas collaborative filtering requires sufficient interaction data to represent items meaningfully.

These complementary strengths and weaknesses motivate the next step of this work: combining collaborative filtering and content-based filtering into a hybrid recommender system that aims to retain the accuracy benefits of collaborative filtering while improving fairness and robustness through content-based information.

4. Hybrid Recommender System (FWLS)

After analysing the content-based (CB) and collaborative filtering (CF) models separately, the next step was to combine them into a hybrid recommender. The main challenge was making both models compatible, since they originally operated in different spaces: the CB model was defined only over the items available in the *extended_games* dataset, whereas the CF model was trained on a larger, global interaction space.

4.1. Defining a Common Item Space

To define a common item universe, we used the CB item set (i.e., the *extended_games* items) as the reference space. This choice ensures that the hybrid recommender can always produce scores for items with content information, aligned with the item representations used in the CB pipeline. This means that both CB and CF have score matrices with the same shape: $(n_{\text{users}} \times n_{\text{CB-items}})$, where users are the test-in (fold-in) users and columns correspond to CB items in a fixed order.

4.1.1. Projecting CF Scores onto the CB Space. Since the CF model is trained on a larger, global interaction space that includes more items than those used by the content-based model, we need to make both of them compatible for the hybrid model to work properly.

To do this, first, we take the CF recommendation scores in the global space that were already computed. Then, we restrict these scores to the content-based item universe by keeping only the columns corresponding to items that exist in this space. At the same time, we select only the rows corresponding to the users in the test set.

This means translating test users and content-based items to their corresponding positions in the global CF score matrix and then selecting the relevant rows and columns. Items that do not exist in the global interaction space simply receive a score of zero in the projected matrix. Finally, we remove items that the user has already interacted with by setting their scores to zero. This ensures that the hybrid recommender only suggests new items, and that both CF and CB scores are fully compatible before being combined.

4.2. Hybrid Score Combination with FWLS

We combine collaborative filtering (CF) and content-based (CB) scores using Feature-Weighted Linear Stacking (FWLS), following Falk's chapter on hybrid recommender systems. In this approach, each component model produces its scores independently, and the final hybrid score is obtained as a weighted linear combination. For a user u and an item i, the hybrid score is defined as:

$$\text{Score}_{\text{hyb}}(u,i) = \alpha \cdot \text{Score}_{\text{CF}}(u,i) + (1-\alpha) \cdot \text{Score}_{\text{CB}}(u,i).$$

The parameter α controls the balance between both components: low values give more weight to content-based information, while high values prioritise collaborative filtering.

4.2.1. Normalization. Following Falk's chapter, we apply per-user min-max normalization to both CF and CB score matrices before combining them. In our implementation, for each user u, we take the non-zero scores in that user's row and rescale them to the interval $[0,1]$ using:

$$\tilde{s}_{u,i} = \frac{s_{u,i} - \min(\mathbf{s}_u)}{\max(\mathbf{s}_u) - \min(\mathbf{s}_u)}.$$

This ensures that, for each user, the lowest scored candidate becomes 0 and the highest becomes 1, making CF and CB scores comparable within the same user. This step is crucial because the two models naturally produce scores on different scales. Without normalization, one component could dominate the hybrid due to larger score magnitudes rather than better ranking quality.

4.3. Selecting α

To select an appropriate value of α, we evaluate a set of values between 0 and 1 (in steps of 0.1). For each α, we:

```
alpha=0.00 | ndcg@20=0.00702 | recall@20=0.01075 | item_gini@20=0.80303
alpha=0.10 | ndcg@20=0.01890 | recall@20=0.02679 | item_gini@20=0.80764
alpha=0.20 | ndcg@20=0.02875 | recall@20=0.03827 | item_gini@20=0.81472
alpha=0.30 | ndcg@20=0.04473 | recall@20=0.05919 | item_gini@20=0.82710
alpha=0.40 | ndcg@20=0.07147 | recall@20=0.09877 | item_gini@20=0.85036
alpha=0.50 | ndcg@20=0.13701 | recall@20=0.21098 | item_gini@20=0.90731
alpha=0.60 | ndcg@20=0.18800 | recall@20=0.28489 | item_gini@20=0.96162
alpha=0.70 | ndcg@20=0.21220 | recall@20=0.29710 | item_gini@20=0.95857
alpha=0.80 | ndcg@20=0.22683 | recall@20=0.30090 | item_gini@20=0.95808
alpha=0.90 | ndcg@20=0.22954 | recall@20=0.30129 | item_gini@20=0.95900
alpha=1.00 | ndcg@20=0.22812 | recall@20=0.29952 | item_gini@20=0.95936
```

Figure 3. Results of the iterative evaluation

NDCG (accuracy)	Recall (accuracy)	User Coverage (fairness)	Item Coverage (diversity)
0.2772	0.3249	0.9179	0.1193

Figure 4. Codabench Results of the Hybrid model

- compute the hybrid score matrix using the weighted sum shown above
- generate top-N recommendations per user from the hybrid scores
- evaluate the resulting recommendations using NDCG@20, Recall@20 and Item Gini@20

This iterative evaluation allows us to directly observe how the balance between CF and CB affects both accuracy and fairness.

As we can observe, the best results are obtained for a high value of α ($\alpha = 0.9$). This is consistent with the CF and CB previous results: collaborative filtering provides most of the predictive power in terms of accuracy, while the content-based component plays a supporting role. When we decrease α and give more weight to the CB component, fairness improves slightly (i.e., the Item Gini decreases), since CB tends to spread recommendations across a broader set of items instead of concentrating them on the most popular games. In practice, this means there is a small accuracy-fairness shift: higher α maximises NDCG/Recall, whereas lower α offers a modest gain in fairness. Finally, CB also helps stabilise the recommendation space in sparse or cold-start situations, where CF scores may be missing or less reliable for some items. Finally, we generated the final recommendation lists and submitted them to Codabench for evaluation.

The accuracy metrics confirm the strong predictive power of the CF-dominated hybrid. In terms of fairness and diversity, the hybrid recommender also exhibits some item diversity. Overall, the hybrid approach balances the strengths of collaborative filtering and content-based methods, providing a solid baseline for the post-hoc re-ranking strategies analysed next.

4.4. Post-hoc Re-ranking Strategies

Once the hybrid recommender was implemented, we explored whether the quality of the recommendation lists could be further improved without retraining the model. For this

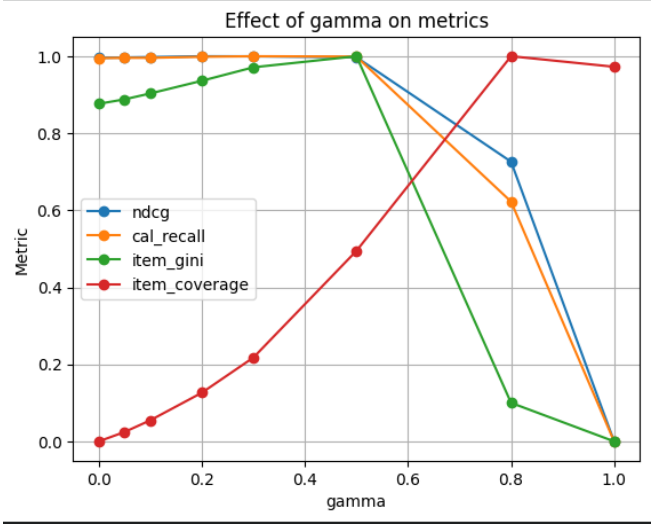


Figure 5. Different values for γ used in Popular penalty

purpose, we applied *post-hoc re-ranking* strategies. Post-hoc methods operate on already computed recommendation scores and aim to adjust the final ranking in order to improve properties such as fairness or diversity, while preserving most of the relevance learned by the base model.

In this work, we focus on two well-known post-hoc techniques: Popularity Penalty and Maximal Marginal Relevance (MMR).

4.4.1. Popularity Penalty. Popularity-based bias is a common issue in collaborative and hybrid recommender systems, as frequently interacted items tend to dominate recommendation lists. Popularity-aware re-ranking methods mitigate this effect by explicitly penalising items that are overly popular, thereby improving fairness and exposure for less popular items. In our implementation, item popularity is computed from the training data as the number of interactions associated with each item. This popularity vector is normalised to ensure comparable magnitudes across items, and then used as a penalty term in the final scores.

For a user u and an item i, the re-ranked score is computed as:

$$\text{Score}_{\text{pop}}(u,i) = \text{Score}_{\text{hyb}}(u,i) - \gamma \cdot \text{Pop}(i),$$

where $\text{Pop}(i)$ represents the normalised popularity of item i and γ controls the strength of the penalty.

We tried varying the values of γ to analyse its effect on the evaluation metrics. Since NDCG, calibrated recall, item Gini and item coverage are defined on different scales, we first applied a min-max normalisation to each metric independently. This normalisation rescales all metrics to the range $[0, 1]$, allowing them to be plotted together in a single figure and compared.

After normalisation, the results show that as γ increases, item coverage steadily improves, indicating that less popular items are increasingly exposed in the recommendation lists. At the same time, accuracy metrics such as NDCG and calibrated recall remain relatively stable for small and moderate values of γ, and only start to decline noticeably for very large penalties.

NDCG (accuracy)	Recall (accuracy)	User Coverage (fairness)	Item Coverage (diversity)
0.2767	0.3252	0.9179	0.1392

Figure 6. Codabench results for Popular Penalty

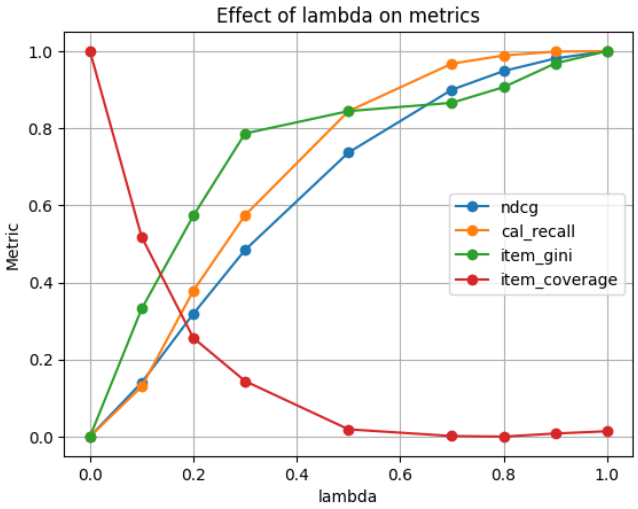


Figure 7. Different values for λ used in MMR

We uploaded to Codabench the results for $\gamma = 0.4$, an intermediate penalty. WE observe that, *item coverage* increases, which indicates a clear improvement in diversity compared to the hybrid baseline. Hence, the popularity penalty reduces a bit the dominance of popular games and gives more exposure to less popular items, without meaningfully harming accuracy.

4.4.2. Maximal Marginal Relevance (MMR). MMR follows a different approach by encouraging diversity within each recommendation list. Instead of selecting items based only on their relevance for the user, MMR also considers how similar another item can be to those already selected. At each step, the next recommendation is chosen by balancing how relevant it is with how different it is from the items that have already been recommended.

$$\text{MMR}(i) = \lambda \cdot \text{Rel}(u,i) - (1-\lambda) \cdot \max_{j \in S} \text{Sim}(i,j).$$

We then analysed the effect of the MMR parameter λ, which controls the balance between relevance and diversity within each recommendation list. As in the popularity penalty analysis, we also applied a min–max normalisation to each metric independently.

After normalisation, the results show a clear trend: for low values of λ, where diversity is prioritised more strongly, accuracy metrics such as NDCG and calibrated recall drop noticeably. As λ increases and more weight is given to

NDCG (accuracy)	Recall (accuracy)	User Coverage (fairness)	Item Coverage (diversity)
0.2647	0.3216	0.9179	0.1179

Figure 8. Codabench results for MMR

relevance, both accuracy metrics improve steadily and approach the values of the hybrid baseline. At the same time, item coverage decreases, indicating that enforcing diversity more aggressively leads to recommendation lists that are less focused on accuracy but more varied.

We uploaded to Codabench the results corresponding to a high value of λ, where relevance is prioritised over diversity. In this setting, MMR achieves reasonable accuracy, but still underperforms both the hybrid model and the popularity penalty approach.

4.4.3. Comparison with the Hybrid Model. The hybrid model achieves the strongest overall accuracy and serves as a robust baseline. Popularity penalty slightly improves fairness and item coverage while keeping accuracy almost unchanged, making it an effective fine-tuning method.

MMR, on the other hand, enforces diversity more strongly but leads to a larger drop in accuracy metrics. As a result, although MMR successfully increases diversity, it is less suitable when accuracy is the primary objective.

Overall, these results confirm that post-hoc re-ranking methods are best understood as adjustment tools rather than replacements for the hybrid model. Popularity penalty provides the most balanced improvement in this setting, whereas MMR highlights the trade-off between relevance and diversity more explicitly.

5. Conclusion

In this work, we studied hybrid recommender systems that combine collaborative filtering and content-based methods, as well as post-hoc re-ranking strategies aimed at improving fairness and diversity. One of the most difficult tasks was making CF and CB compatible: aligning users, items and score matrices into a common space required significantly more effort than initially expected.

The hybrid recommender proved to be more stable and reliable than standalone collaborative or content-based models, confirming that combining both sources of information helps mitigate their individual weaknesses, as expected. In addition, applying a popularity penalty as a post-hoc re-ranking strategy led to a modest but consistent improvement in diversity, increasing item coverage while.

At the same time, some results were less pronounced than anticipated. The gains from post-hoc re-ranking were relatively small, and MMR in particular did not improve performance as much as we thought. This behaviour shows that post-hoc methods are not designed to dramatically outperform the base model in terms of accuracy, but rather to tune the balance between relevance, fairness and diversity.

For future work we could explore alternative re-ranking strategies, such as switched or context-aware ensemble recommenders, to further improve the results.

References

[1] F. Falk, *Content-Based Filtering*. In: Recommender Systems Handbook. Springer, 2015.

[2] F. Falk, *Taking the Best of All Algorithms: Implementing Hybrid Recommenders*. In: Recommender Systems Handbook. Springer, 2015.

[3] J. Li, Y. Zhang, and A. Smutny, "A Survey on Post-hoc Recommendation Fairness and Diversity," *ACM Computing Surveys*, 2021.